

UNIVERSIDADE VEIGA DE ALMEIDA – UVA

Engenharia de Software

Paradigmas de Linguagens de Programação

Atividade Avaliativa Final

Problema 1 – Sistema de Notas e Médias de Alunos

Membros do Grupo:

Nome Completo	RA
Pedro Domingues Fernandes	1260106125
Leandro Viana de Oliveira Filho	1260101932
Wellerson Hilário Flor de Sá	1260109007

Professor: Rogerio Mandelli

Turma: 4169EGSN1A1_P1 | Campus: Barra Marapendi

Rio de Janeiro, 2026

1. Paradigmas de Programação e Linguagens Utilizadas

1.1 Paradigma Estruturado

O paradigma estruturado é uma das abordagens mais consolidadas da computação. Baseia-se na decomposição do problema em funções e procedimentos independentes, organizados de forma sequencial e modular. O fluxo de controle é definido por estruturas como if/else e for, evitando o uso de saltos incondicionais.

Neste trabalho, o paradigma estruturado foi implementado em Python. A linguagem é de alto nível, dinamicamente tipada e de propósito geral, tornando-a ideal para demonstrar o estilo estruturado por meio de funções e procedimentos bem organizados.

Características principais exploradas na implementação:

- Modularização por funções independentes: `calcular_media`, `determinar_situacao`, `calcular_estatisticas_turma` e `ordenar_alunos_por_nome`
- Separação clara entre entrada de dados, processamento e exibição
- Uso de estruturas de controle (for, if/else) para iterar e decidir
- Representação dos alunos como dicionários Python, passados explicitamente entre funções

1.2 Paradigma Orientado a Objetos (POO)

O paradigma orientado a objetos organiza o software em torno de objetos que encapsulam dados (atributos) e comportamentos (métodos). Os principais conceitos são: encapsulamento, herança, polimorfismo e abstração.

A implementação POO também foi realizada em Python, utilizando a classe `Alunos` para encapsular os dados de cada estudante e os comportamentos associados. Essa abordagem permite que cada aluno seja tratado como uma entidade autônoma, com estado e lógica próprios.

Características principais exploradas na implementação:

- Definição da classe `Alunos` com atributos `nome` e `notas`
- Método `media_aluno()` para calcular a média individualmente
- Método `aprovado()` que encapsula a regra de aprovação (média $\geq 6,0$)
- Uso de `*args` no construtor para aceitar número variável de notas
- Ordenação de objetos com `sorted()` e `lambda` como critério de comparação

2. Descrição do Problema

2.1 Contexto

O problema escolhido foi o Problema 1 – Sistema de Notas e Médias de Alunos. O sistema simula um cenário acadêmico em que um professor precisa registrar as notas de seus alunos, calcular médias individuais, determinar a situação de cada estudante e obter estatísticas gerais da turma. Trata-se de um problema clássico de processamento de dados, simples o suficiente para ser resolvido em diferentes paradigmas, mas rico o bastante para evidenciar as diferenças de abordagem entre eles.

2.2 Requisitos

- Armazenar nome e notas de n alunos
- Calcular a média individual de cada aluno
- Determinar a situação de cada aluno (Aprovado ou Reprovado)
- Exibir a listagem ordenada dos alunos
- Calcular a média geral da turma
- Identificar a maior nota registrada na turma
- Identificar a menor nota registrada na turma

2.3 Objetivo da Solução

Demonstrar como o mesmo problema pode ser resolvido de formas estruturalmente distintas conforme o paradigma adotado, evidenciando as vantagens, limitações e características de cada abordagem.

3. Implementação Comentada

3.1 Implementação Estruturada (Python)

A solução estruturada organiza o programa em funções independentes, cada uma responsável por uma tarefa específica. O fluxo principal é controlado pela função principal(), que orquestra a chamada às demais.

A função calcular_media() recebe uma lista de notas e retorna a média aritmética utilizando um laço for. A função determinar_situacao() aplica a regra de negócio e retorna uma string com a situação do aluno. A função calcular_estatisticas_turma() percorre todos os alunos já processados para calcular a média geral, a maior e a menor nota. A função ordenar_alunos_por_nome() retorna uma nova lista ordenada alfabeticamente usando sorted() com uma função lambda como chave.

A decisão de usar dicionários Python para representar cada aluno é característica do paradigma estruturado: os dados são passados explicitamente entre as funções, sem encapsulamento de estado. Cada função recebe o que precisa como argumento e retorna o resultado processado.

3.2 Implementação Orientada a Objetos (Python)

A solução POO define a classe Alunos, que encapsula nome e notas como atributos de instância. Os métodos media_aluno() e aprovado() representam comportamentos da entidade, eliminando a necessidade de passar dados explicitamente para funções externas.

O construtor recebe o nome e notas variáveis (*notas), convertidas internamente para uma lista. A ordenação é feita com sorted() aplicado à lista de objetos, usando lambda aluno: aluno.media_aluno() como critério. O loop de exibição itera sobre os objetos ordenados, chamando seus métodos diretamente.

A principal diferença em relação ao código estruturado é que, no POO, os dados e as operações sobre eles vivem no mesmo lugar (a classe), enquanto no estruturado os dados trafegam como argumentos entre funções separadas. Isso torna o código POO mais coeso e fácil de escalar.

4. Análise Comparativa

A tabela a seguir resume a comparação entre os dois paradigmas nos principais critérios avaliados:

Critério	Estruturado (Python)	Orientado a Objetos (Python)
Legibilidade	Alta	Alta
Modularização	Boa (funções)	Excelente (classes)
Reutilização	Média	Alta
Facilidade de manutenção	Boa	Excelente
Desempenho percebido	Alto	Alto
Produtividade	Alta	Média
Facilidade de implementação	Alta	Média
Tratamento de exceções	Manual	Encapsulado na classe

4.1 Legibilidade

Ambos os códigos apresentam alta legibilidade. O estruturado tem fluxo linear e intuitivo, próximo do raciocínio sequencial natural. O OO agrupa dados e comportamentos na classe, o que facilita a compreensão do que cada aluno é e pode fazer. Para desenvolvedores com experiência em OO, o segundo é ainda mais claro.

4.2 Modularização e Reutilização

O Orientado a Objetos oferece melhor modularização: a classe `Alunos` pode ser importada e reutilizada em qualquer sistema sem modificações. O estruturado também é modularizado por funções, mas depende de dicionários com estrutura específica, o que cria um acoplamento implícito entre as funções que precisam conhecer as chaves do dicionário.

4.3 Tipos de Dados, Escopo e Armazenamento

Nos dois paradigmas, Python adota tipagem dinâmica — os tipos são inferidos em tempo de execução. No estruturado, os dados são representados como dicionários e trafegam entre funções como argumentos, com escopo local a cada função. No OO, os dados são encapsulados em atributos de instância (`self.nome`, `self.notas`), com escopo de objeto, acessíveis por qualquer método da classe.

4.4 Persistência de Dados

Nenhuma das implementações realiza persistência em arquivo ou banco de dados — os dados são declarados diretamente no código. Em uma evolução futura, o OO facilitaria a integração com bancos de dados relacionais via ORM (como SQLAlchemy), pois cada instância da classe corresponderia naturalmente a um registro da tabela. O estruturado exigiria serialização manual dos dicionários.

4.5 Facilidade de Implementação

O paradigma estruturado foi mais direto de implementar, pois seu raciocínio é sequencial e não exige planejamento prévio de estruturas. O OO demandou uma etapa inicial de modelagem da classe antes da codificação, mas o resultado final é mais organizado e extensível. Para sistemas maiores, o investimento inicial do OO se paga com facilidade de manutenção.

5. Conclusão

A realização desta atividade permitiu ao grupo vivenciar na prática as diferenças entre os paradigmas de programação estudados ao longo da disciplina, aplicando-os a um mesmo problema concreto.

O paradigma estruturado mostrou-se o mais adequado para problemas de pequena escala e processamento linear, como o proposto. Sua legibilidade e facilidade de implementação o tornam uma escolha natural para scripts e sistemas simples, onde a criação de classes seria desnecessária.

O paradigma orientado a objetos demonstrou seu potencial de organização e escalabilidade. Para um sistema real com múltiplas turmas, histórico de notas e diferentes tipos de avaliação, a abordagem OO seria claramente superior, pois facilita a manutenção, a extensão do código e a integração com outros sistemas.

As principais dificuldades encontradas foram relacionadas à decisão de quais responsabilidades atribuir à classe no paradigma OO e como equilibrar modularização com simplicidade no paradigma estruturado.

Os principais aprendizados consolidados nesta atividade foram: a compreensão de que não existe um paradigma universalmente superior, mas sim abordagens mais ou menos adequadas a cada contexto; e a importância de conhecer múltiplos paradigmas para escolher a ferramenta certa para cada problema.